Maria Rita Piekas e Tales Cruz da Silva

Avaliação de Desempenho de uma Aplicação CRUD em Sistemas Distribuídos

Santa Maria – RS

2026

Maria Rita Piekas e Tales Cruz da Silva

Avaliação de Desempenho de uma Aplicação CRUD em Sistemas Distribuídos

Relatório apresentado à disciplina de Sistemas Distribuídos, como parte das atividades de implementação e avaliação de desempenho de uma aplicação CRUD.

Universidade Federal de Santa Maria
Curso de Ciência da Computação

Santa Maria – RS
2026

Sumário

1 INTRODUÇÃO 3

2 IMPLEMENTAÇÃO E METODOLOGIA 4
2.1 Arquitetura 4
2.2 Preparação dos testes 4
2.3 Experimento com índice 5

3 RESULTADOS E DISCUSSÃO 6
3.1 Testes de carga 6
3.2 Efeito do índice em cidade 6
3.3 SQL e NoSQL 7

4 CONCLUSÃO 8

REFERÊNCIAS 9

1 Introdução

Este trabalho apresenta a implementação e a avaliação inicial de uma aplicação CRUD (*Create, Read, Update, Delete*) para gerenciamento de clientes. A aplicação utiliza Python e Flask para disponibilizar uma API REST, PostgreSQL para armazenamento persistente e Docker Compose para executar os componentes em contêineres. O Apache JMeter foi utilizado para simular requisições concorrentes.

Foram definidos três perfis de carga: A, com 50% de leituras e 50% de escritas; B, com 75% de leituras e 25% de escritas; e C, com 25% de leituras e 75% de escritas. Os testes apresentados foram realizados com uma base inicial de 50.000 registros. Também foi analisado o efeito de um índice em consultas por cidade. A repetição dos testes com 20.000 registros permanece como etapa futura.

2 Implementação e metodologia

2.1 Arquitetura

A arquitetura possui três componentes: o cliente ou gerador de carga, o servidor Flask/Gunicorn e o banco PostgreSQL. O cliente envia requisições HTTP ao servidor, que executa operações de consulta ou alteração no banco. Os componentes foram executados em contêineres Docker na mesma máquina virtual Linux; o JMeter também foi executado nesse ambiente.

As principais tecnologias foram Python, Flask, PostgreSQL, Docker, Docker Compose e Apache JMeter. A API oferece as rotas apresentadas na Tabela 1.

Tabela 1 – Principais rotas da API

Método	**Rota**	**Operação**
GET	`/api/clientes`	Lista clientes, com paginação e filtros.
GET	`/api/clientes/{id}`	Consulta um cliente pelo identificador.
POST	`/api/clientes`	Cadastra um cliente.
PUT	`/api/clientes/{id}`	Atualiza um cliente.
DELETE	`/api/clientes/{id}`	Remove um cliente.

A tabela de clientes contém campos como identificador, nome, e-mail, CPF, cidade e idade. A chave primária identifica cada registro, enquanto restrições de unicidade para e-mail e CPF ajudam a preservar a integridade dos dados. A rota `/health` permite verificar se o servidor está respondendo.

2.2 Preparação dos testes

Um script Python gerou 50.000 registros sintéticos em aproximadamente 2,37 segundos. A quantidade foi conferida com uma consulta `SELECT COUNT(*)`. Cada cenário foi executado com 100 clientes concorrentes e 1.000 requisições, por meio de planos JMeter no formato JMX.

Tabela 2 – Distribuição das requisições por cenário

Cenário	**GET**	**POST**	**Total**
A (50/50)	500	500	1.000
B (75/25)	750	250	1.000
C (25/75)	250	750	1.000

Após cada execução, os arquivos JTL foram verificados para conferir a distribuição dos métodos HTTP e as respostas. Como POST cria registros, as inclusões extras foram removidas antes do cenário seguinte, de modo que cada teste começasse com 50.000 registros. Os indicadores considerados foram latência média, tempos mínimo e máximo, throughput (requisições por segundo) e taxa de erros.

2.3 Experimento com índice

Para observar o efeito de um índice sobre consultas por cidade, foi executada a consulta abaixo antes e depois da criação de um índice adicional:

```
EXPLAIN ANALYZE
SELECT * FROM clientes
WHERE cidade = 'Santa Maria';

CREATE INDEX idx_clientes_cidade
ON clientes (cidade);
```

O comando `EXPLAIN ANALYZE` permite observar o plano escolhido pelo PostgreSQL e o tempo de execução da consulta. Essa medição é distinta dos testes de carga do JMeter.

3 Resultados e discussão

3.1 Testes de carga

A Tabela 3 resume os resultados obtidos com a base inicial de 50.000 registros.

Tabela 3 – Resultados dos testes de carga

Cenário	GET	POST	Média	Mín.	Máx.	Req./s
A	500	500	6 ms	2 ms	90 ms	99,9
B	750	250	5 ms	2 ms	52 ms	100,3
C	250	750	7 ms	2 ms	121 ms	99,6

O JMeter não registrou erros nos três cenários. O throughput permaneceu próximo de 100 requisições por segundo. O cenário B apresentou a menor latência média (5 ms), enquanto o cenário C apresentou a maior média (7 ms) e o maior tempo máximo (121 ms). Esse resultado é compatível com a hipótese de que uma proporção maior de escritas possa elevar o custo de processamento, mas os dados de uma única execução não demonstram causalidade nem superioridade estatística.

A contagem de registros também correspondeu ao número de inclusões: após o cenário B, a base passou temporariamente a 50.250 registros; antes do cenário C, voltou a 50.000; e, após C, chegou a 50.750. Esse controle é importante para manter condições iniciais comparáveis.

O percentil 95 (P95), que ajuda a caracterizar os tempos de resposta mais elevados, ainda não foi extraído dos arquivos JTL. Além disso, os testes foram curtos e executados na mesma máquina virtual que hospedava a aplicação e o banco. Portanto, os valores devem ser interpretados como resultados iniciais desse ambiente, e não como uma previsão direta de desempenho em produção.

3.2 Efeito do índice em cidade

Sem o índice adicional, o PostgreSQL utilizou uma varredura sequencial (`Seq Scan`) para consultar os registros cuja cidade era Santa Maria. A consulta retornou 3.338 registros e apresentou tempo de execução de 3,747 ms. Após a criação de `idx_clientes_cidade`, o plano passou a utilizar `Bitmap Index Scan` e `Bitmap Heap Scan`, com tempo de 3,259 ms.

A redução observada foi de aproximadamente 13,02%, calculada por:

$$\frac{3{,}747 - 3{,}259}{3{,}747} \times 100 \approx 13{,}02\%.$$

Essa diferença corresponde a uma consulta isolada e pode variar conforme o estado dos caches, a carga do sistema e a distribuição dos dados. Índices podem acelerar consultas seletivas, mas ocupam espaço e precisam ser atualizados em operações de escrita. Assim, o benefício deve ser avaliado considerando tanto as leituras quanto o custo de manutenção.

3.3 SQL e NoSQL

O PostgreSQL é adequado ao modelo utilizado, que possui atributos definidos e restrições de integridade, como a unicidade de e-mail e CPF. Bancos NoSQL abrangem modelos distintos — por exemplo, documentos, chave-valor e grafos — e podem ser úteis quando os requisitos de estrutura, distribuição ou escalabilidade favorecem essas soluções.

Não é possível afirmar que SQL ou NoSQL seria mais rápido para esta aplicação com base nos experimentos realizados, pois não foi implementada uma versão equivalente em um banco NoSQL. Uma comparação válida exigiria manter o conjunto de dados, as operações, a carga e as condições de execução tão semelhantes quanto possível, além de considerar consistência, transações e integridade.

4 Conclusão

Foi implementada uma API CRUD com Flask e PostgreSQL, executada em contêineres Docker e submetida a três cenários de carga com 100 clientes concorrentes e 1.000 requisições por cenário. Com 50.000 registros iniciais, os testes terminaram sem erros e apresentaram throughput próximo de 100 requisições por segundo. O cenário com 75% de leituras teve a menor latência média, enquanto o cenário com 75% de escritas registrou a maior média e o maior tempo máximo.

Na consulta filtrada por cidade, o índice adicional reduziu o tempo observado de 3,747 ms para 3,259 ms, aproximadamente 13,02%. Como se trata de uma medição isolada, ainda é necessário repetir a análise e avaliar o índice sob carga. Também permanecem pendentes a extração do P95, a execução dos cenários com 20.000 registros e uma eventual comparação experimental com uma solução NoSQL.

Os resultados mostram a importância de analisar o perfil de acesso, a integridade dos dados, os planos de consulta e as condições de execução antes de tirar conclusões sobre desempenho ou escolher uma tecnologia.

Referências

[1] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. Disponível em: https://www.postgresql.org/docs/. Acesso em: 12 out. 2026.

[2] APACHE SOFTWARE FOUNDATION. *Apache JMeter User's Manual*. Disponível em: https://jmeter.apache.org/usermanual/. Acesso em: 12 out. 2026.

[3] PALLETS PROJECTS. *Flask Documentation*. Disponível em: https://flask.palletsprojects.com/. Acesso em: 12 out. 2026.

[4] DOCKER INC. *Docker Documentation*. Disponível em: https://docs.docker.com/. Acesso em: 12 out. 2026.